

Atividade 3 - Compressão, Representação e Descrição

Aluno: Vitor Fontenele de Oliveira Linhares - 1700778

Repositório

Imports

```
In [65]: import numpy as np
import cv2
import matplotlib.pyplot as plt
from pathlib import Path
import pandas as pd
```

Setup

```
In [66]: plt.rcParams['image.cmap'] = 'gray'

IMG1_PATH = Path('mars.jpg')
IMG2_PATH = Path('universe.jpg')
OUT_DIR = Path('outputs')

def load_image(path):
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(f'Imagem não encontrada: {path.resolve()}')
    bgr = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if bgr is None:
        raise ValueError(f'Não foi possível ler a imagem: {path.resolve()}')
    if bgr.ndim == 2:
        return bgr
    return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
```

```
def to_gray(img):
    if img.ndim == 2: return img
    return (0.299 * img[:, :, 0] + 0.587 * img[:, :, 1] + 0.114 * img[:, :, 2]).astype(np.float32)

def save_image(filename, img, out_dir=OUT_DIR):
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    out_path = out_dir / filename
    arr = np.clip(img, 0, 255).astype(np.uint8)
    if arr.ndim == 3:
        arr = cv2.cvtColor(arr, cv2.COLOR_RGB2BGR)
    cv2.imwrite(str(out_path), arr)
    return out_path

def show_grid(images, titles, cols, figsize=None):
    n = len(images)
    rows = (n + cols - 1) // cols

    if figsize is None:
        figsize = (cols * 4, rows * 4)

    _, axes = plt.subplots(rows, cols, figsize=figsize)
    axes = axes.flatten() if n > 1 else [axes]

    for i in range(len(axes)):
        if i < n:
            axes[i].imshow(images[i], vmin=images[i].min(), vmax=images[i].max())
            axes[i].set_title(titles[i])
            axes[i].axis('off')

    plt.tight_layout()
    plt.show()

def initialize():
    mars = load_image(IMG1_PATH)
    mars_gray = to_gray(mars)
    universe = load_image(IMG2_PATH)
    universe_gray = to_gray(universe)

    save_image('mars_gray.jpg', mars_gray, '.')
```

```
save_image('universe_gray.jpg', universe_gray, '.')  
show_grid([mars, mars_gray, universe, universe_gray], ['Marte', 'Marte Cinza', 'Universo', 'Universo Cinza'], c  
initialize()
```

Marte



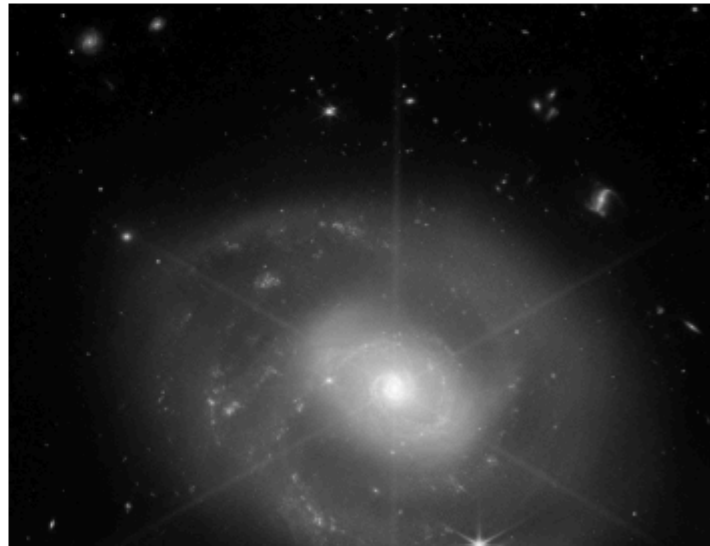
Marte Cinza



Universo



Universo Cinza





Questão 1

Funções auxiliares

```
In [67]: def get_matrix_transpose(matrix):
        return matrix.T

def get_basis_matrix(N):
    T = np.zeros((N, N), dtype=np.float32)
    for u in range(N):
        for x in range(N):
            if u == 0:
                T[u, x] = 1.0 / np.sqrt(N)
            else:
                T[u, x] = np.sqrt(2.0 / N) * np.cos((2 * x + 1) * u * np.pi / (2.0 * N))
    return T

def dct(block, T, T_transposed):
    return T @ block @ T_transposed

def icdt(dct_block, T_transposed, T):
    return T_transposed @ dct_block @ T

def get_quantization_matrix(quality, quantization_matrix):
    if quality <= 0: quality = 1
    if quality > 100: quality = 100
    scale = (5000 / quality) if quality < 50 else (200 - 2 * quality)
    Q = np.floor((quantization_matrix * scale + 50) / 100)
    Q[Q == 0] = 1
    return Q.astype(np.float32)

def quantize(dct_block, Q):
```

```

    return np.round(dct_block / Q)

def dequantize(block_quant, Q):
    return block_quant * Q

```

Função orquestradora e de comparação

```

In [68]: def compress(img_gray, quality, block_size):
    H, W = img_gray.shape
    H_new = H - (H % block_size)
    W_new = W - (W % block_size)
    img_trimmed = img_gray[:H_new, :W_new].copy()

    img_reconstructed = np.zeros_like(img_trimmed)
    img_quantized_coefficients = np.zeros_like(img_trimmed)

    T = get_basis_matrix(block_size)
    T_transposed = get_matrix_transpose(T)

    base_q_matrix = np.array([
        [16, 11, 10, 16, 24, 40, 51, 61],
        [12, 12, 14, 19, 26, 58, 60, 55],
        [14, 13, 16, 24, 40, 57, 69, 56],
        [14, 17, 22, 29, 51, 87, 80, 62],
        [18, 22, 37, 56, 68, 109, 103, 77],
        [24, 35, 55, 64, 81, 104, 113, 92],
        [49, 64, 78, 87, 103, 121, 120, 101],
        [72, 92, 95, 98, 112, 100, 103, 99]
    ], dtype=np.float32)

    Q = get_quantization_matrix(quality, base_q_matrix)

    for r in range(0, H_new, block_size):
        for c in range(0, W_new, block_size):
            block = img_trimmed[r:r+block_size, c:c+block_size]

            block_centered = block - 128.0

            block_dct = dct(block_centered, T, T_transposed)

```

```

        block_quant = quantize(block_dct, Q)
        img_quantized_coefficients[r:r+block_size, c:c+block_size] = block_quant

        block_dequant = dequantize(block_quant, Q)
        block_idct = icdt(block_dequant, T_transposed, T)

        img_reconstructed[r:r+block_size, c:c+block_size] = block_idct + 128.0

    return img_trimmed, img_reconstructed, img_quantized_coefficients

def compress_evaluation(original, reconstructed, quantized_coefs):
    mse = np.mean((original - reconstructed) ** 2)
    psnr = 100.0 if mse == 0 else 10 * np.log10((255.0 ** 2) / mse)
    sparsity_pct = (np.sum(quantized_coefs == 0) / quantized_coefs.size) * 100

    return mse, psnr, sparsity_pct

```

Rodando

```

In [69]: quality_values = [95, 50, 20, 5, 1]
         results = []
         block=8

         mars_gray = load_image('mars_gray.jpg')
         H, W = mars_gray.shape
         orig_trimmed = mars_gray[:H-(H%block), :W-(W%block)]

         reconstructed_images = [orig_trimmed]
         titles = ['Original']

         for q in quality_values:
             _, recon_img, coefs = compress(mars_gray, quality=q, block_size=block)

             mse, psnr, sparsity = compress_evaluation(orig_trimmed, recon_img, coefs)

             results.append({
                 'Qualidade': q,

```

```
        'MSE': mse,  
        'PSNR': psnr,  
        'Sparsity': sparsity  
    })  
  
    reconstructed_images.append(recon_img)  
    titles.append(f'Qualidade = {q}')  
  
show_grid(reconstructed_images, titles, cols=3)  
  
df_results = pd.DataFrame(results)  
display(df_results)
```


Original



Qualidade = 95



Qualidade = 50



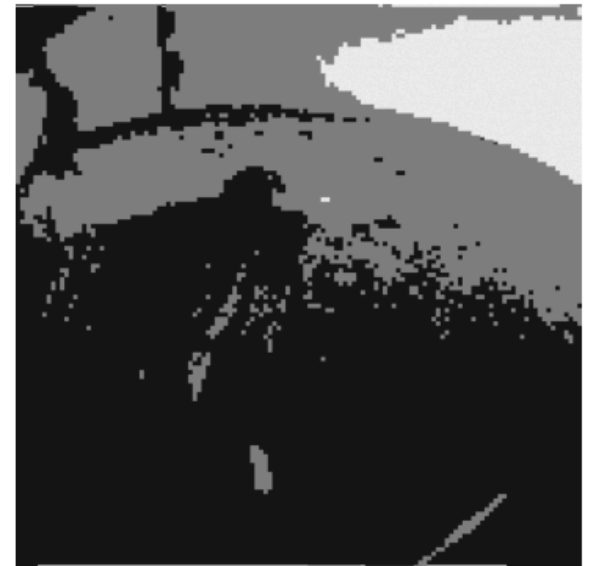
Qualidade = 20



Qualidade = 5



Qualidade = 1



	Qualidade	MSE	PSNR	Sparsity
0	95	0.497266	51.164918	84.371758
1	50	7.480867	39.391284	89.315319
2	20	15.450534	36.241369	96.070957
3	5	47.672278	31.348145	98.244953
4	1	108.349358	27.782540	98.919392

Análise

O impacto visual das transformações nas imagens reconstruídas revela uma degradação progressiva conforme os níveis de compressão se tornam mais severos.

- $Q = 95$ e $Q = 50$: O algoritmo preserva uma fidelidade quase total em relação à imagem original. As perdas ficam restritas a detalhes texturais praticamente imperceptíveis. Mesmo na imagem de menor qualidade conseguimos identificar detalhes no céu e no solo.
- ($Q = 20$): Mesmo sob uma compressão considerável, a imagem mantém integridade estrutural de seus principais elementos. Os contornos da sonda, a linha do horizonte e o solo continuam perfeitamente reconhecíveis. Contudo, as texturas perdem o aspecto natural e passam a exibir transições mais grosseiras e ruidosas. Começam a aparecer sinais de degradação na parte superior direita do horizonte e na parte inferior da figura.
- $Q = 5$ a $Q = 1$: Em $Q = 5$, a pixelização se torna dominante na imagem. O colapso mais drástico do experimento ocorre justamente no salto de $Q = 5$ para $Q = 1$. No nível mais baixo ($Q = 1$), a geometria e as formas dos objetos são completamente destruídas. O que resta na imagem são macroblocos sólidos de tons de cinza que apenas sinalizam as grandes massas de iluminação da cena (a região clara do céu ao fundo e o sombreamento geral do solo).

Um ponto interessante observado no grid é que as distorções se tornam evidentes primeiro na região do céu.

Além disso, um detalhe curioso de se notar no céu é a preservação do que aparenta ser um planeta localizado no canto esquerdo superior. Embora não haja resolução alguma para definir feições ou detalhes geométricos precisos, o algoritmo consegue delimitar que existe um corpo ou objeto ali

Os dados comparativos extraídos consolidam as análises visuais e revelam o fato de que existem dinâmicas na compressão que operam de forma puramente matemática antes de se manifestarem visualmente. Na transição de $Q = 95$ para $Q = 50$, a taxa de esparsidade passa de 84.37% para 89.31%. Ao reduzir para $Q = 20$, essa taxa dá um salto expressivo e atinge **96.07%**. O ponto é que, visualmente, não se nota extrema diferença na qualidade das imagens $Q = 50$ e $Q = 20$. Isso demonstra a eliminação de redundâncias, o algoritmo limpou 96% dos dados da matriz sem causar um impacto destrutivo equivalente na imagem observada.

Questão 2

Classe Descritora e suas rotinas internas

```
In [70]: class Descriptor:
    def __init__(self, img_gray):
        self.img_arr = img_gray.astype(np.float32)
        self.N_pixels = self.img_arr.size

        self.global_metrics = self._compute_global()
        self.spatial_metrics, self.diff_H, self.diff_V = self._compute_spatial()

    def _compute_global(self):
        media = np.mean(self.img_arr)

        variancia = np.mean((self.img_arr - media) ** 2)

        contagem, _ = np.histogram(self.img_arr, bins=256, range=(0, 256))
        probabilidades = contagem / self.N_pixels
        energia = np.sum(probabilidades ** 2)

        return media, variancia, energia

    def _compute_spatial(self):
        diff_H = np.abs(self.img_arr[:, :-1] - self.img_arr[:, 1:])

        diff_V = np.abs(self.img_arr[:-1, :] - self.img_arr[1:, :])

        media_diff_H = np.mean(diff_H)
        media_diff_V = np.mean(diff_V)
```

```
return (media_diff_H, media_diff_V), diff_H, diff_V
```

Função orquestradora

```
In [71]: def descriptor_analysis(blocks, names):
    results = []
    imgs = []
    titles = []

    for bloco, nome in zip(blocks, names):
        descriptors = Descriptor(bloco)

        med, var, ene = descriptors.global_metrics
        df_h, df_v = descriptors.spatial_metrics

        results.append({
            'Região': nome,
            'Média': round(med, 2),
            'Variância': round(var, 2),
            'Energia': round(ene, 5),
            'ΔH': round(df_h, 2),
            'ΔV': round(df_v, 2)
        })

        imgs.append(bloco)
        imgs.append(descriptors.diff_H)
        imgs.append(descriptors.diff_V)

        titles.append(f"{nome}")
        titles.append("ΔH (Horizontal)")
        titles.append("ΔV (Vertical)")

    df_comparativo = pd.DataFrame(results)
    display(df_comparativo.style.hide(axis='index'))

    show_grid(imgs, titles, cols=3)
```

Definindo imagens e blocos

```
In [72]: img_universo = load_image('universe_gray.jpg')
img_marte = load_image('mars_gray.jpg')

print(f"Universo: {img_universo.shape} (Altura, Largura)")
print(f"Marte: {img_marte.shape} (Altura, Largura)")

uni_bloco1 = img_universo[300:500, 300:500]
uni_descr1 = Descriptor(uni_bloco1)

uni_bloco2 = img_universo[300:500, 0:200]
uni_descr2 = Descriptor(uni_bloco2)

mar_bloco1 = img_marte[700:900, 200:400]
mar_descr1 = Descriptor(mar_bloco1)

mar_bloco2 = img_marte[100:300, 500:700]
mar_descr2 = Descriptor(mar_bloco2)

grid_universo = [
    img_universo, uni_bloco1,
    img_universo, uni_bloco2
]

titulos_universo = [
    "Universo", "Subcorte 1",
    "Universo", "Subcorte 2"
]

show_grid(grid_universo, titulos_universo, cols=2)

grid_marte = [
    img_marte, mar_bloco1,
    img_marte, mar_bloco2
]

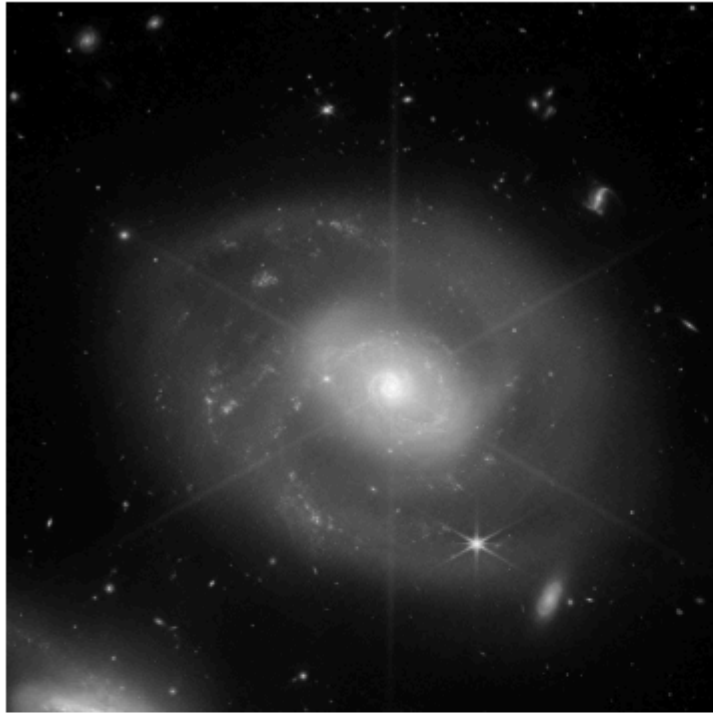
titulos_marte = [
```

```
    "Marte", "Subcorte 1",  
    "Marte", "Subcorte 2"  
]  
  
show_grid(grid_marte, titulos_marte, cols=2)
```

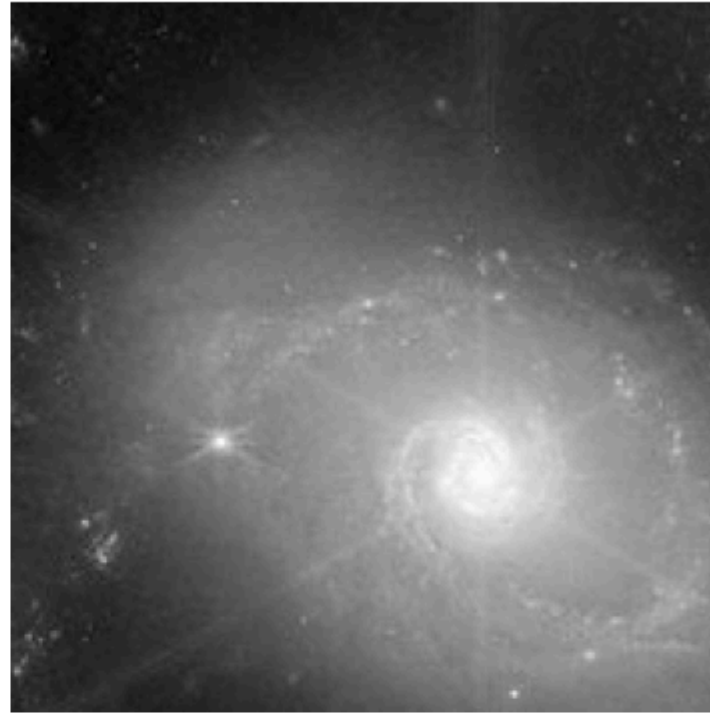
Universo: (797, 800) (Altura, Largura)

Marte: (1024, 1024) (Altura, Largura)

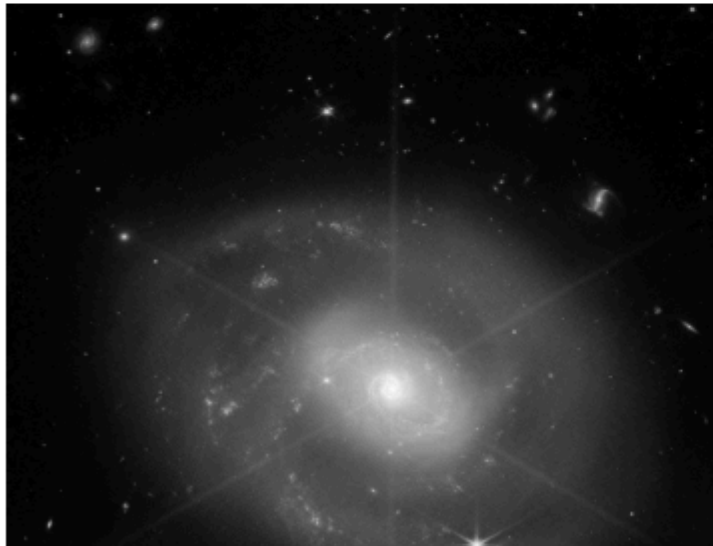
Universo



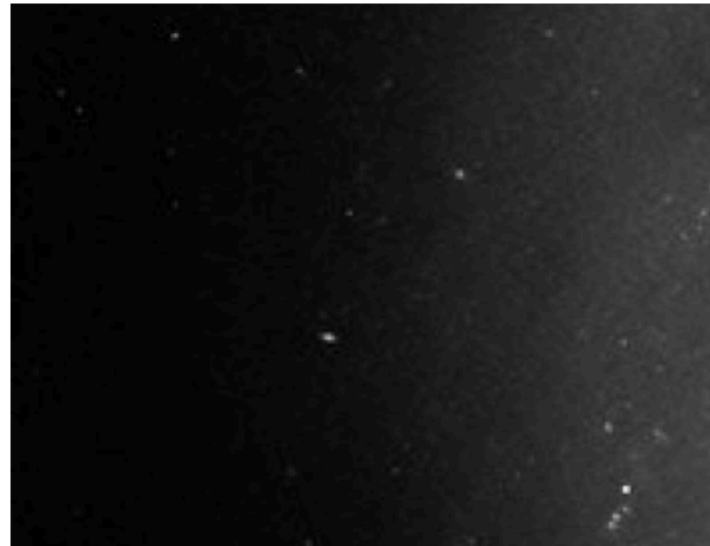
Subcorte 1



Universo



Subcorte 2

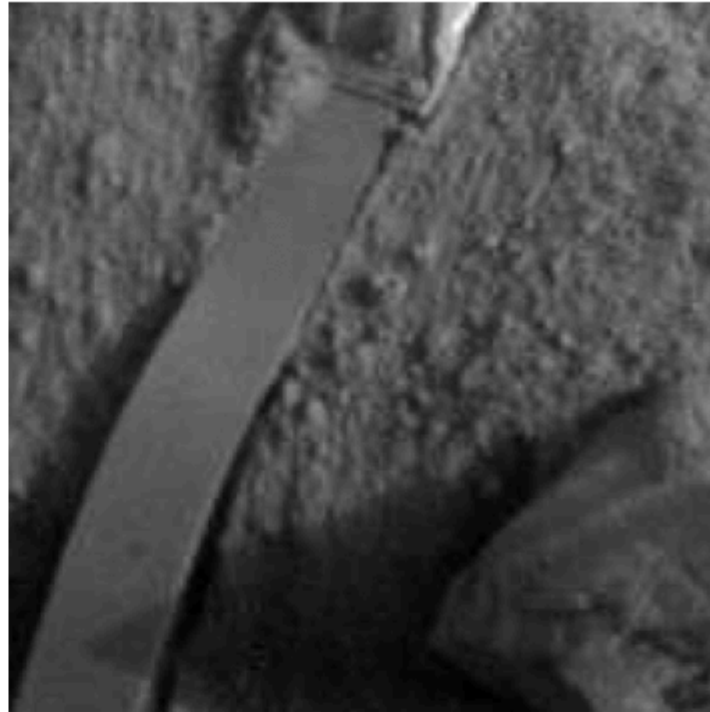




Marte



Subcorte 1

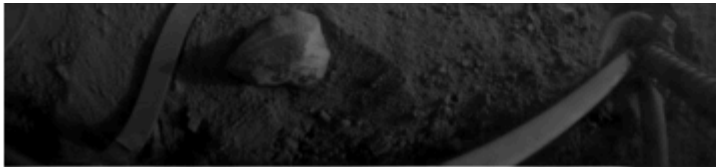


Marte



Subcorte 2



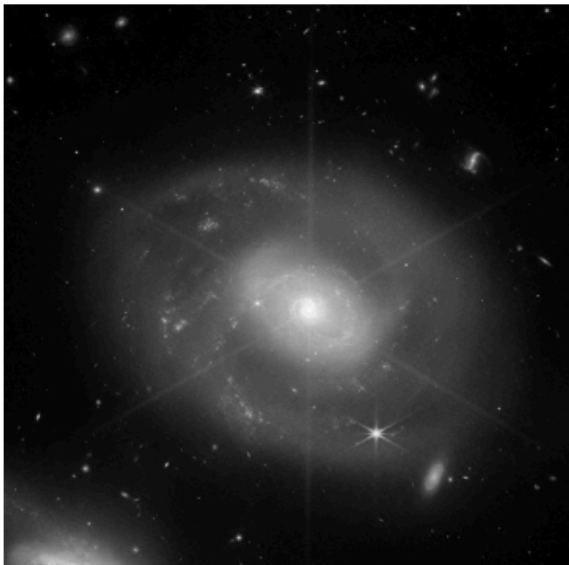


Rodando

```
In [73]: descriptor_analysis(  
    [img_universo, uni_bloco1, uni_bloco2],  
    ["Universo", "Subcorte 1 (Centro)", "Subcorte 2 (Lateral-Esq)"]  
)  
  
descriptor_analysis(  
    [img_marte, mar_bloco1, mar_bloco2],  
    ["Marte", "Subcorte 1 (Solo/Sonda)", "Subcorte 2 (Horizonte)"]  
)
```

Região	Média	Variância	Energia	ΔH	ΔV
Universo	36.080002	1551.359985	0.025980	1.900000	1.990000
Subcorte 1 (Centro)	128.229996	1735.339966	0.007920	2.610000	2.800000
Subcorte 2 (Lateral-Esq)	21.410000	299.079987	0.042710	1.460000	1.520000

Universo



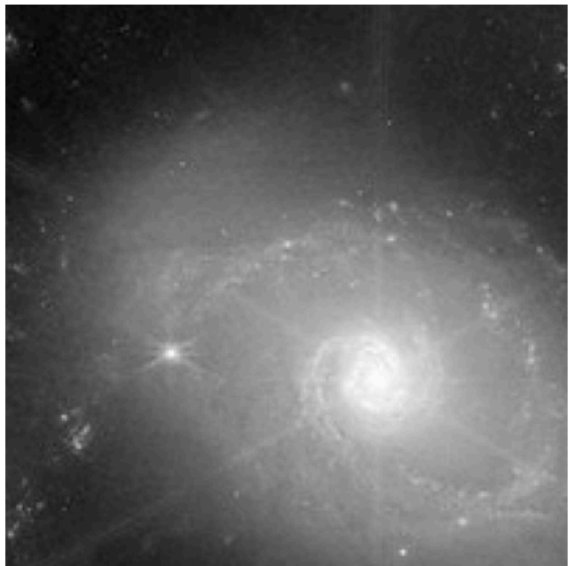
ΔH (Horizontal)



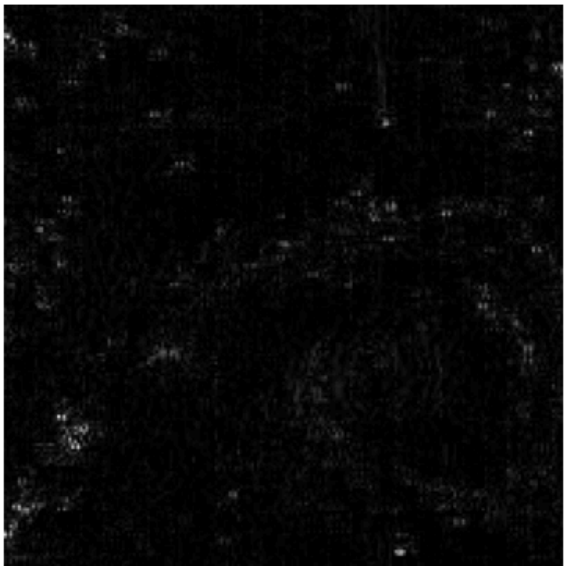
ΔV (Vertical)



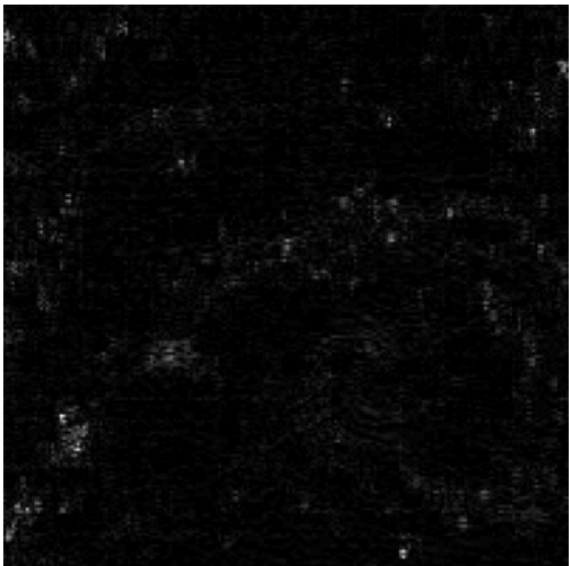
Subcorte 1 (Centro)



ΔH (Horizontal)



ΔV (Vertical)



Subcorte 2 (Lateral-Esq)

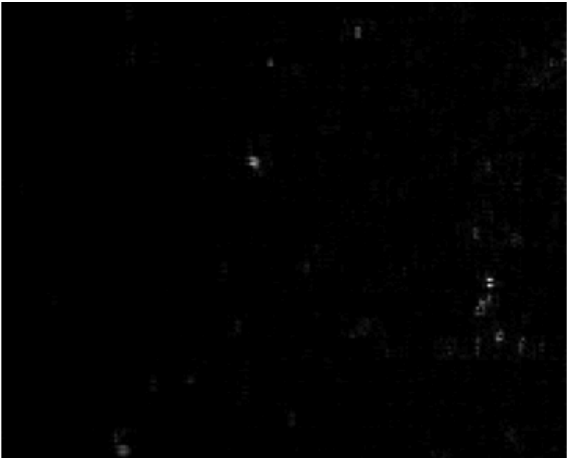
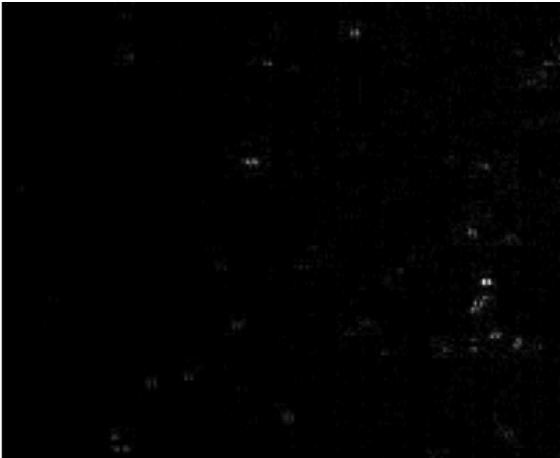
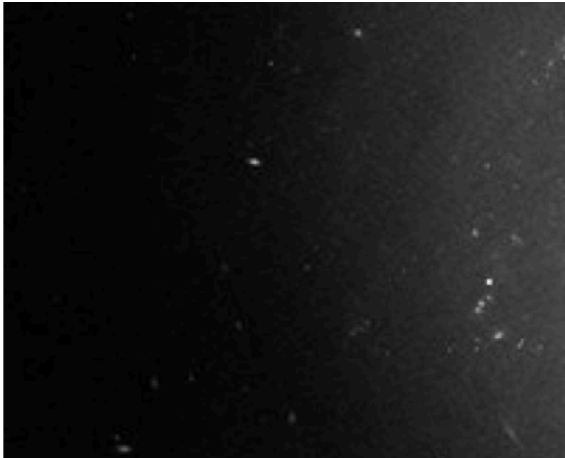


ΔH (Horizontal)



ΔV (Vertical)



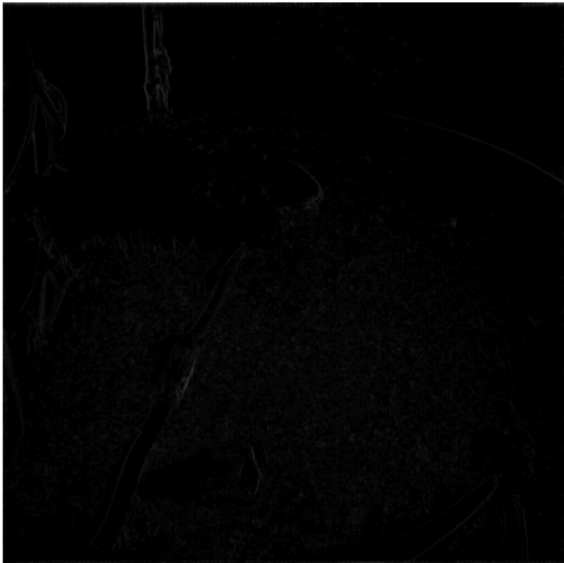


Região	Média	Variância	Energia	ΔH	ΔV
Marte	81.330002	2783.560059	0.007870	2.790000	2.870000
Subcorte 1 (Solo/Sonda)	41.650002	299.160004	0.017940	3.570000	2.640000
Subcorte 2 (Horizonte)	128.639999	2298.449951	0.019180	1.460000	2.310000

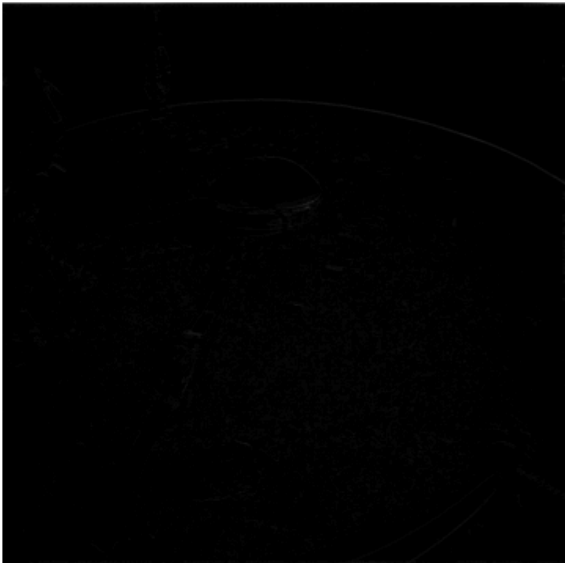
Marte



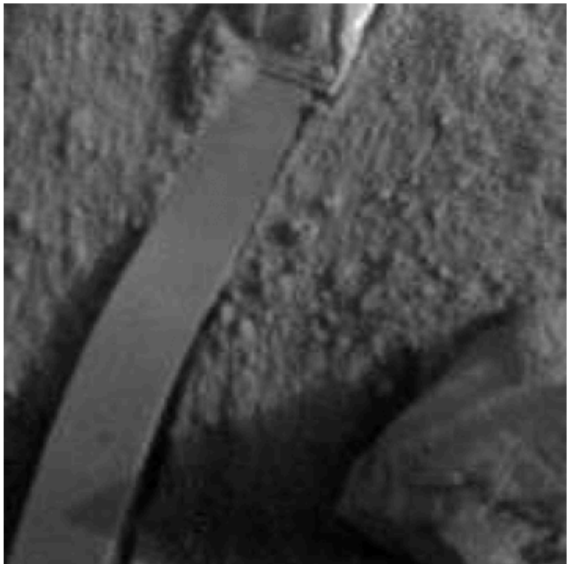
ΔH (Horizontal)



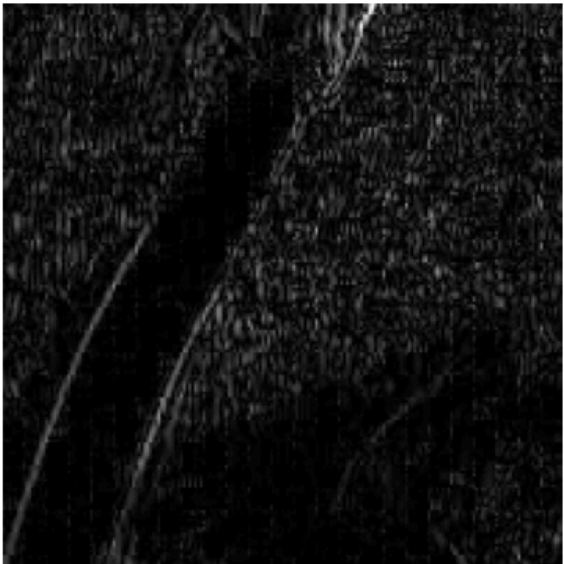
ΔV (Vertical)



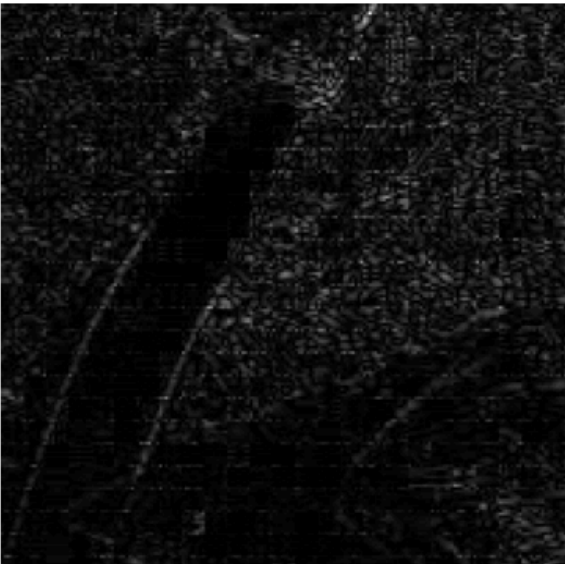
Subcorte 1 (Solo/Sonda)



ΔH (Horizontal)



ΔV (Vertical)



Subcorte 2 (Horizonte)



ΔH (Horizontal)



ΔV (Vertical)





Análise

Imagem do Universo

A caracterização do cenário astronômico revela como os descritores respondem a diferentes níveis de entropia e densidade estrutural dentro de um ambiente predominantemente escuro.

A imagem completa estabelece a linha de base do experimento com uma média de **36.08**, o que explicita o predomínio do plano de fundo escuro do espaço profundo. No entanto, ao isolar o Subcorte 1 (Centro), onde se concentram o núcleo galáctico e estruturas estelares de alta intensidade, o brilho médio eleva-se para **128.23**. O comportamento da variância (**1735.34**) e da energia (**0.00792**) no centro reflete uma alta entropia, indicando que a ampla distribuição de tons pelo histograma gera alto contraste e baixa uniformidade. No Subcorte 2 (Lateral-Esquerda), o cenário se inverte, pois a dominância de tons escuros e contínuos afunila o histograma, reduzindo a variância para **299.08** e fazendo a energia saltar para **0.04271**, o que comprova a transição para uma região de menor complexidade visual.

No domínio espacial, a alternância abrupta entre os corpos celestes brilhantes e o fundo escuro no Subcorte 1 gera descontinuidades na matriz, elevando as médias de variação para **2.61** (ΔH) e **2.80** (ΔV), o que caracteriza um padrão de textura de alta frequência. Já no Subcorte 2, a transição entre pixels vizinhos ocorre de forma sutil através do degradê suave na periferia da galáxia, reduzindo as variações locais para **1.46** (ΔH) e **1.52** (ΔV).

Imagem de Marte

O cenário da superfície marciana apresenta uma dinâmica textural que evidencia que métricas de variabilidade global nem sempre se traduzem em rugosidade espacial local.

No Subcorte 1 (Solo/Sonda), que compreende a região inferior do cenário, as projeções de sombra reduzem a média de brilho para **41.65** e limitam a variância global em **299.16**. Contudo, a alternância entre os grãos de areia, pedras e os componentes metálicos da sonda introduz microbordas constantes no domínio espacial. Essa flutuação ponto a ponto faz com que os descritores espaciais aumentem, registrando o maior índice horizontal do experimento ($\Delta H = 3.57$) e confirmando que regiões escuras e de baixo contraste global podem conter alta frequência de textura local.

O Subcorte 2 (Horizonte), focado na transição entre o céu e o solo de Marte, exibe um comportamento estatístico singular. A média se eleva para **128.64** devido à luminosidade do céu, e a variância atinge **2298.45**, sendo este o maior valor do experimento. Esse alto contraste macroscópico ocorre porque o bloco é dividido em duas massas tonais opostas, correspondentes ao céu claro e ao solo escuro. Todavia, o descritor horizontal ($\Delta H = 1.46$) permanece baixo porque caminhar lateralmente dentro do céu ou dentro do solo revela superfícies localmente homogêneas. O aspecto crítico reside no descritor vertical ($\Delta V = 2.31$), cujo aumento em relação ao componente horizontal quantifica o degrau de intensidade gerado ao cruzar a linha física do horizonte, o que evidencia a propriedade de anisotropia da borda.

A imagem completa de Marte sintetiza esses dois comportamentos ao registrar uma média de **81.33** e uma variância de **2783.56**, refletindo o contraste geral do cenário. No entanto, suas métricas espaciais ($\Delta H = 2.79$ e $\Delta V = 2.87$) operam de forma amortecida, pois a suavidade do céu limpo compensa a rugosidade do solo pedregoso na contabilidade global da matriz.

Relação com Aplicações em Classificação de Imagens

O conjunto formado por esses cinco descritores constitui um vetor de características que atua como uma assinatura matemática capaz de reduzir a dimensionalidade de uma imagem para fins de processamento.

Em sistemas de aprendizado de máquina, algoritmos como KNN ou SVM utilizam esses padrões para segmentar o espaço computacional. Blocos que agrupam alta variância e baixas diferenças espaciais são mapeados como transições suaves ou planos de fundo, enquanto blocos de altos deltas espaciais e baixa energia são categorizados como texturas complexas. Essa modelagem possui aplicação direta em sistemas de navegação autônoma de robôs, para a detecção de terrenos trafegáveis e obstáculos rochosos, e em sistemas de sensoriamento remoto por satélite.